Roadmap de Desarrollo — UPB-CIENTÍFICA

0. Principio rector

La razón de fondo detrás de todo este roadmap es una sola: **en un sistema distribuido, el orden de construcción no lo dicta la lista de features, lo dicta el grafo de dependencias de tiempo de ejecución.** Un servicio no se puede probar de forma creíble si el servicio del que depende para autenticar, almacenar o comunicar no existe todavía, aunque "en papel" ambos aparezcan como ítems independientes del backlog. Por eso primero se construye lo que todos los demás necesitan para funcionar (identidad, comunicación, almacenamiento), y solo después lo que consume esas bases (HPC, sync, multimedia), y al final lo que observa y pulen el conjunto (monitoreo, clientes avanzados, optimización).

---

1. Roadmap por fases

Fase 1 — Análisis

Objetivo: fijar contratos antes de escribir una línea de código de negocio. Definir qué hace cada servicio, qué NO hace, y con qué protocolo habla con quién (WSDL de Auth, `.proto` de gRPC, interfaces RMI). Esta fase produce los "contratos" que permiten que 5 personas trabajen en paralelo sin pisarse.

Por qué primero: en SOA, el contrato es el límite de responsabilidad entre equipos. Si dos desarrolladores empiezan a codear sin haber acordado el contrato, el retrabajo de integración se dispara — es el error más caro y más común en proyectos SOA estudiantiles.

Fase 2 — Diseño

Objetivo: arquitectura por capas de cada servicio, modelo de datos, diagramas de secuencia de los flujos críticos (auth, job HPC, sync), decisión de qué corre en qué máquina física.

Por qué después del análisis: el diseño interno de capas depende de tener claro el contrato externo (Fase 1); diseñar capas antes de fijar el contrato obliga a rediseñar cuando el contrato cambie.

Fase 3 — Infraestructura base

Objetivo: máquinas/VMs del Centro de Computación Avanzada, red interna, resolución de nombres, cuentas de servicio, repositorios de código, CI básico (compilación + pruebas), sin Docker/K8s/Cloud.

Por qué antes del desarrollo de servicios: ningún servicio puede desplegarse ni probarse de forma realista sin red y máquinas ya operativas. Desarrollar "en local" únicamente oculta problemas de comunicación distribuida que aparecerán tarde y mal si se posponen.

Fase 4 — Desarrollo (por oleadas, ver sección 2)

Objetivo: construir los servicios en el orden de dependencia real, no alfabético ni por "lo más interesante primero".

Fase 5 — Integración

Objetivo: conectar servicios ya probados unitariamente y validar los flujos punta a punta (usuario → auth → servicio → storage → respuesta).

Fase 6 — Pruebas de sistema

Objetivo: carga, concurrencia, tolerancia a fallos, seguridad — con el sistema completo, no por servicio aislado.

Fase 7 — Despliegue y cierre

Objetivo: despliegue estable en el CCA, documentación final, manuales de usuario y operación.

---

2. Orden de implementación y justificación técnica

El orden correcto **no** es el propuesto por defecto en el enunciado ("Directorio Activo → Auth → API → Repositorio → Shared File → Sync → Photos → Streaming → Scheduler → HPC → Monitoreo → Clientes → Optimización"). Ese orden casi acierta, pero comete un error grave: **deja Scheduler y HPC casi al final**, cuando HPC es el corazón de negocio del sistema (es la razón de ser de "UPB-CIENTÍFICA") y depende de Storage, no de Photos/Streaming. Retrasarlo maximiza el riesgo de que el equipo se quede sin tiempo para la parte más difícil y más importante del proyecto.

Orden recomendado (corregido)

1. **Infraestructura base** (red, máquinas, repos, CI)
2. **Directorio Activo + Authentication Service (PHP/SOAP)**
3. **Distributed Storage Service** (incluye Repository físico y metadatos)
4. **Scheduler** (aislado, con jobs simulados/mock, sin MPI real todavía)
5. **Cluster MPI + HPC Service (Java/RMI)** — integración real Scheduler↔MPI
6. **Shared File Service**
7. **File Sync Service (Go/gRPC)**
8. **Photo Album Service**
9. **Streaming Service**
10. **Monitoring Service**
11. **Clientes** (Web primero, luego Windows/Linux, luego Android/iOS)
12. **Optimización y endurecimiento (hardening)**

Justificación paso a paso

- **1 → 2:** Auth necesita máquinas y red antes de existir como servicio corriendo. Y Auth debe ir primero entre los servicios de negocio porque **todos los demás lo consumen** para autorizar cada operación; si Auth no existe, ningún otro servicio puede probarse con seguridad real (solo con mocks, lo cual retrasa la detección de problemas de integración).
- **2 → 3:** Storage no necesita HPC ni Sync para existir, pero **sí necesita Auth** para saber de quién son los archivos y aplicar cuotas/permisos. Se construye antes que cualquier servicio que "guarda algo", porque Shared File, Photos, Streaming, Sync y HPC (resultados) todos dependen de él.
- **3 → 4:** El Scheduler es lógica de planificación pura (colas, asignación, estados) que **puede y debe** probarse con trabajos simulados antes de acoplarlo a MPI real. Separar esta etapa reduce muchísimo el riesgo: si algo falla, sabes si es el Scheduler o es MPI, en lugar de depurar ambos a la vez.
- **4 → 5:** Solo cuando el Scheduler ya funciona con jobs de prueba tiene sentido conectar el clúster MPI real. Aquí es donde se integra Java/RMI con la ejecución paralela real — es la parte más técnica y riesgosa del proyecto, por eso se hace con el Scheduler ya validado y no antes.
- **5 → 6/7:** Shared File y Sync son funcionalmente independientes de HPC (ambos solo necesitan Auth + Storage), así que **podrían ir en paralelo con la fase de HPC**, pero se listan después en la secuencia principal porque HPC es la prioridad de negocio y consume al equipo más senior; con 5 desarrolladores, en la práctica esto se paraleliza (ver sección 7).
- **8 → 9:** Photos y Streaming son las funcionalidades de menor prioridad de negocio (son "servicios adicionales" según el enunciado original) y comparten patrón con Shared File, así que van después, reutilizando el mismo patrón de acceso a Storage ya probado.
- **10 (Monitoring) va después de tener servicios reales que observar:** construir Monitoring antes es desperdiciar esfuerzo — no hay nada que monitorear todavía, y sus dashboards no se pueden validar sin datos reales de Scheduler/MPI/Storage.
- **11 (Clientes) al final de cada backend, no al final de todo el proyecto:** los clientes deben desarrollarse **incrementalmente en paralelo**, apenas cada servicio expone su contrato (ver Fase de Análisis) — no esperar a que todo el backend esté terminado. Lo que sí va estrictamente al final es la versión **pulida y completa** de todos los clientes.
- **12 (Optimización) al final:** rendimiento y hardening solo tienen sentido cuando la funcionalidad ya es correcta; optimizar prematuramente es la causa clásica de retrabajo.

---

3. Dependencias por servicio

Servicio	Depende de	Es dependencia de	Debe existir antes de comenzarlo
Authentication Service	Infraestructura, Directorio Activo	Todos los demás servicios	Red interna, máquina de auth
Distributed Storage Service	Authentication Service, Infraestructura	Shared File, Sync, Photos, Streaming, HPC (resultados)	Auth funcionando (permisos/cuotas)
Scheduler	Authentication Service (opcional en mock)	HPC Service	Modelo de estados de job definido (Fase de Diseño)
Cluster MPI	Scheduler	HPC Service	Scheduler validado con jobs simulados
HPC Service	Auth, Scheduler, Cluster MPI, Storage (resultados)	Clientes, Monitoring	Scheduler + MPI integrados
Shared File Service	Auth, Storage	Clientes	Storage funcionando
File Sync Service	Auth, Storage	Clientes de escritorio	Storage con versionado de metadatos
Photo Album Service	Auth, Storage	Clientes	Storage funcionando
Streaming Service	Auth, Storage	Clientes	Storage funcionando (contenido multimedia)
Monitoring Service	Todos los servicios anteriores (observa métricas reales)	Ninguno (es terminal)	Al menos Scheduler, MPI y Storage corriendo
Clientes	Auth + el servicio específico que consumen	Ninguno	Contrato (WSDL/proto/RMI) publicado del servicio correspondiente

4. MVP (Producto Mínimo Viable)

Definición del MVP: el conjunto mínimo que demuestra la promesa central del negocio — *"un investigador se autentica, sube un trabajo científico, y obtiene un resultado ejecutado en paralelo sobre el clúster."*

Servicios incluidos en el MVP:

1. Authentication Service (mínimo: login + token, sin roles avanzados)
2. Distributed Storage Service (mínimo: guardar/leer archivos, sin replicación avanzada)
3. Scheduler (cola simple, un algoritmo de asignación básico)
4. Cluster MPI (ejecución real, aunque sea con 2-3 nodos)
5. HPC Service (envío de job, consulta de estado, descarga de resultado)
6. Un cliente mínimo (Web es el más rápido de iterar)

Por qué este subconjunto y no otro: Shared File, Sync, Photos, Streaming y Monitoring son **valor agregado**, no la propuesta de valor central. Un MVP con Photos pero sin HPC funcionando no demuestra nada sobre "UPB-CIENTÍFICA"; un MVP con HPC funcionando, aunque sea sin sincronización de archivos, sí demuestra que la arquitectura completa (SOA + capas + comunicación distribuida + paralelismo real) funciona de extremo a extremo. Esto también reduce el riesgo del proyecto: si el HPC end-to-end falla, es mejor descubrirlo en la semana 6 que en la semana 14.

---

5. Evaluación del orden propuesto en el enunciado

El orden sugerido era:

> Infraestructura → Directorio Activo → Autenticación → API → Repositorio → Shared File → Sync → Photo Album → Streaming → Scheduler → HPC → Monitoreo → Clientes → Optimización

Es correcto en su primera mitad (Infra → Directorio → Auth → API → Repositorio), pero tiene un problema estructural: coloca **Scheduler y HPC casi al final**, después de Shared File, Sync, Photos y Streaming. Esto invierte la prioridad de negocio y de riesgo:

- HPC/Scheduler son técnicamente lo más difícil (concurrencia real, paralelismo, RMI) — dejarlo para el final significa enfrentarlo con menos tiempo y menos margen de error.
- Photos, Streaming y Sync son variaciones del mismo patrón ("guardar y servir contenido desde Storage") — son de bajo riesgo técnico comparativo y no aportan nada a la prueba de que el "sistema HPC" en sí funciona.

Orden corregido propuesto: el de la sección 2 (Storage → Scheduler → MPI/HPC antes de Shared File/Sync/Photos/Streaming). Monitoreo, Clientes y Optimización sí quedan correctamente al final en ambos esquemas.

6. Riesgos por fase

Infraestructura

- **Técnico:** red mal configurada entre nodos del clúster → *mitigación:* validar conectividad y latencia antes de escribir código de servicios.
- **Diseño:** subestimar cuántas máquinas/puertos se necesitan → *mitigación:* inventario de puertos y protocolos desde la Fase de Análisis.

Autenticación

- **Seguridad:** tokens sin expiración o sin firma → *mitigación:* JWT o tokens firmados con TTL corto desde el diseño inicial, no como parche después.
- **Integración:** cada servicio interpretando permisos distinto → *mitigación:* modelo de roles/permisos único, documentado y probado con casos de contrato antes de que otros equipos lo consuman.

Storage

- **Escalabilidad:** metadatos centralizados como cuello de botella → *mitigación:* diseñar el índice de metadatos pensando en réplica desde el inicio, aunque el MVP no la implemente.
- **Sincronización:** condiciones de carrera al escribir el mismo archivo desde dos clientes → *mitigación:* versionado optimista desde el diseño.

Scheduler / HPC / MPI

- **Rendimiento:** mala asignación de nodos degrada todo el clúster → *mitigación:* probar el Scheduler con carga simulada antes de acoplar MPI real.
- **Concurrencia:** condiciones de carrera entre Workers → *mitigación:* pruebas de estrés con jobs concurrentes desde etapas tempranas, no solo al final.
- **Integración:** RMI es sensible a versiones de JVM y a fallos de red silenciosos → *mitigación:* timeouts explícitos y reintentos, probados con "cortes de red" simulados.

Sync / Shared File / Photos / Streaming

- **Diseño:** resolución de conflictos de sync mal definida desde el inicio es muy cara de cambiar después → *mitigación:* fijar la política de resolución de conflictos en Fase de Diseño, no improvisarla en desarrollo.
- **Rendimiento:** streaming compite por ancho de banda con sync/HPC → *mitigación:* pruebas de red compartida antes de integración final.

Monitoreo

- **Diseño:** monitorear métricas que nadie usa → *mitigación:* definir con el equipo qué decisiones se tomarán con cada métrica antes de implementarla.

Clientes

- **Integración:** clientes construidos contra una versión vieja del contrato → *mitigación:* congelar el contrato (WSDL/proto/RMI) antes de que el equipo de clientes empiece, y versionarlo si cambia.

---

7. Paralelización con 6 desarrolladores

Con un sexto integrante conviene **separar lo que antes estaba forzosamente junto por falta de gente**: Shared File/Sync se separa de Photos/Streaming, y Monitoring se separa de Clientes. Eso reduce la carga por persona y, sobre todo, reduce el "acoplamiento humano" — cuando una sola persona lleva demasiados servicios distintos, se vuelve sin querer un cuello de botella de coordinación.

Integrante	Responsabilidad principal	Depende de	Puede empezar cuando...
Dev A	Infraestructura + Authentication Service (PHP/SOAP)	Nadie	Desde el día 1
Dev B	Distributed Storage Service + Repository	Contrato de Auth (mock al inicio)	Cuando el WSDL de Auth esté definido (no implementado)
Dev C	Scheduler + Cluster MPI + HPC Service (Java/RMI) — pipeline HPC completo	Contrato de Storage (mock al inicio)	Cuando el contrato de Storage esté definido
Dev D	Shared File Service + File Sync Service (Go/gRPC)	Contrato de Storage	En paralelo con Dev C, cuando Storage tenga contrato definido
Dev E	Photo Album Service + Streaming Service	Contrato de Storage	En paralelo con Dev C y Dev D, cuando Storage tenga contrato definido
Dev F	Monitoring Service + Clientes (Web, Windows/Linux, Android/iOS) + pruebas de integración	Contratos de todos (consume, no implementa lógica de negocio)	El cliente Web puede iniciar apenas Auth tenga contrato; Monitoring espera a tener servicios reales corriendo

Por qué este reparto y no otro:

- **Dev C se queda solo con el pipeline HPC completo** (Scheduler + MPI + HPC Service) en lugar de repartirlo, porque son tres piezas fuertemente acopladas entre sí (el

Scheduler solo tiene sentido en función de MPI, y el HPC Service es la fachada RMI de ambos) y es la parte más riesgosa técnicamente — dividirla entre dos personas multiplicaría la coordinación necesaria justo donde menos conviene.

- **Separar Dev D (Shared/Sync) de Dev E (Photos/Streaming)** es posible ahora porque ambos pares son independientes entre sí (solo comparten la dependencia de Storage) — con 5 devs se juntaban por necesidad, con 6 pueden avanzar en paralelo real sin pisarse.
- **Dev F concentra Monitoring + Clientes + pruebas** porque son las piezas que "consumen" al resto del sistema en lugar de implementar lógica de negocio nueva — tiene sentido que una sola persona mantenga la visión transversal de cómo se integra todo, y es quien coordina las pruebas de integración final.

Cómo evitar bloqueos: la clave sigue siendo la misma que con 5 desarrolladores: **nadie espera a que otro termine de implementar — solo a que otro termine de *definir el contrato***. En la semana 1-2 se fijan todos los WSDL/proto/interfaces RMI (Fase de Análisis), y a partir de ahí cada dev trabaja contra mocks de los servicios que consume, integrando de verdad más adelante. Con 6 personas esto es aún más importante, porque ahora hay 3 devs (C, D, E) trabajando en paralelo directamente sobre el contrato de Storage — si ese contrato cambia tarde, el retrabajo se triplica en vez de duplicarse.

Tareas que sí deben esperar (secuenciales, no paralelizables):

- Integración real Scheduler↔MPI (Dev C) no puede probarse de verdad hasta que Storage (Dev B) tenga al menos una versión funcional para guardar resultados.
- Monitoring (Dev F) no puede validarse con datos reales hasta que Scheduler/MPI/Storage estén corriendo.
- Pruebas de integración de sistema completo (todos) solo pueden hacerse cuando todos los servicios del MVP están desplegados.

Qué puede ejecutarse en paralelo desde el día 1 (a partir de la semana 3, con los contratos ya fijados):

- Dev A (Auth) y Dev B (Storage, contra mock de Auth) — en paralelo.
- Una vez Storage tiene contrato: Dev C (HPC), Dev D (Shared/Sync) y Dev E (Photos/Streaming) — los tres en paralelo, cada uno contra un mock de Storage hasta que la versión real esté lista.
- Dev F puede avanzar el cliente Web desde que Auth tiene contrato, y sumar Monitoring en cuanto haya al menos un servicio real desplegado para observar.

---

8. Hitos del proyecto

- **Hito 1:** Infraestructura funcionando (red, máquinas, repos, CI) — todos los devs pueden desplegar algo.

- **Hito 2:** Authentication Service completo (login, token, permisos básicos).
- **Hito 3:** Distributed Storage Service funcionando (guardar/leer con permisos de Auth).
- **Hito 4:** Scheduler validado con jobs simulados (sin MPI real).
- **Hito 5:** Primer trabajo HPC ejecutado de extremo a extremo (Auth → Scheduler → MPI → Storage → resultado). ← **MVP alcanzado.**
- **Hito 6:** Shared File + Sync funcionando sobre Storage.
- **Hito 7:** Photo Album + Streaming funcionando.
- **Hito 8:** Monitoring mostrando métricas reales de todos los servicios.
- **Hito 9:** Clientes Windows/Linux/Android/iOS integrados contra todos los servicios.
- **Hito 10:** Sistema optimizado y endurecido (hardening, pruebas de carga superadas).

---

9. Estrategia de pruebas por fase

Fase	Qué probar	Tipo de prueba	Debe validarse antes de continuar
Infraestructura	Conectividad, latencia entre nodos	Pruebas de red manuales	Todas las máquinas se ven entre sí
Auth	Login, expiración de token, roles	Unitarias + contrato (WSDL)	Un cliente de prueba puede autenticarse y recibir permisos correctos
Storage	CRUD de archivos, cuotas, permisos	Unitarias + integración con Auth	Un archivo subido por un usuario no es visible para otro sin permiso
Scheduler	Encolado, asignación, estados de job	Unitarias + carga simulada (jobs falsos)	El Scheduler no pierde ni duplica jobs bajo concurrencia
MPI/HPC	Ejecución paralela real, timeouts, reintentos	Integración + concurrencia + tolerancia a fallos (matar un worker a mitad de ejecución)	Un job real se ejecuta y el resultado llega íntegro a Storage
Sync/Shared/Photos/Streaming	Conflictos de versión, permisos, streaming bajo carga	Integración + rendimiento	No hay corrupción de datos al sincronizar concurrentemente
Monitoring	Exactitud de métricas vs estado real	Integración	Las métricas coinciden con el estado real de Scheduler/MPI/Storage
Clientes	Consumo correcto de cada contrato	Integración end-to-end	Cada cliente completa el flujo completo (login → acción → resultado)

Fase	Qué probar	Tipo de prueba	Debe validarse antes de continuar
Sistema completo	Carga, concurrencia multiusuario, fallos de red	Pruebas de sistema + estrés	El sistema se degrada con gracia, no se cae en cascada

---

10. Entregables por fase

- **Análisis:** diagramas de casos de uso, contratos (WSDL, `.proto`, interfaces RMI), matriz de dependencias.
- **Diseño:** diagramas de arquitectura por capas, diagramas de secuencia de flujos críticos, modelo de datos por servicio, plan de despliegue físico.
- **Infraestructura:** documentación de red, scripts de aprovisionamiento, repos con CI configurado.
- **Desarrollo:** código de cada servicio, pruebas unitarias, documentación de API.
- **Integración:** ambiente integrado funcionando, pruebas de integración automatizadas, bitácora de incidencias resueltas.
- **Pruebas de sistema:** informe de pruebas de carga/concurrencia/fallos, métricas de rendimiento.
- **Despliegue:** manual de instalación, manual de operación, manual de usuario por tipo de cliente.

---

11. Criterios para avanzar de fase

- **Análisis → Diseño:** todos los contratos (WSDL/proto/RMI) están firmados y documentados; no debe haber ambigüedad sobre qué recibe/devuelve cada operación.
- **Diseño → Infraestructura:** el diagrama de despliegue físico existe y está validado contra la disponibilidad real de máquinas del CCA.
- **Infraestructura → Desarrollo:** todas las máquinas están accesibles entre sí y el pipeline de CI compila un "hola mundo" en cada tecnología (PHP/SOAP, Java/RMI, Go/gRPC).
- **Desarrollo → Integración:** cada servicio pasa sus pruebas unitarias y expone su contrato tal cual fue definido en Análisis (sin desviaciones no documentadas).
- **Integración → Pruebas de sistema:** el flujo MVP completo (Auth → Storage → Scheduler → MPI → resultado) funciona de extremo a extremo al menos una vez sin intervención manual.
- **Pruebas de sistema → Despliegue:** no quedan fallos críticos abiertos (caídas, pérdida de datos); los fallos de rendimiento conocidos están documentados con plan de

mitigación.

---

12. Cronograma sugerido (16 semanas, proyecto semestral)

- **Semana 1-2:** Análisis — casos de uso, contratos, matriz de dependencias.
- **Semana 3:** Diseño — arquitectura por capas, diagramas de secuencia, modelo de datos.
- **Semana 4:** Infraestructura base — red, máquinas, CI.
- **Semana 5-6:** Authentication Service completo.
- **Semana 6-7:** Distributed Storage Service (en paralelo con el cierre de Auth).
- **Semana 7-8:** Scheduler (con jobs simulados).
- **Semana 9-10:** Cluster MPI + HPC Service — integración real. **Hito MVP al final de la semana 10.**
- **Semana 11:** Shared File Service + File Sync Service.
- **Semana 12:** Photo Album Service + Streaming Service.
- **Semana 13:** Monitoring Service.
- **Semana 14:** Clientes (Web, Windows/Linux, Android/iOS) — integración final.
- **Semana 15:** Pruebas de sistema (carga, concurrencia, tolerancia a fallos).
- **Semana 16:** Optimización, hardening, documentación final y entrega.

(Si el semestre es más corto, comprimir sacrificando primero Streaming/Photos, nunca Auth/Storage/HPC — son el núcleo evaluable de "sistema distribuido".)

---

13. Buenas prácticas transversales

Decisiones que deben tomarse antes de escribir código:

- El modelo de permisos/roles (afecta a todos los servicios).
- El protocolo exacto de cada servicio (no cambiar SOAP por REST a mitad de camino).
- La política de resolución de conflictos de Sync.
- El esquema de versionado de metadatos en Storage.

Errores de arquitectura a evitar:

- Meter lógica de negocio en la capa de Presentación/API "por rapidez" — se paga caro al integrar.
- Compartir base de datos entre dos servicios "para ahorrar tiempo" — rompe el bajo acoplamiento y es casi imposible de deshacer después.
- Validar permisos de forma distinta en cada servicio en lugar de reutilizar el mismo modelo de Auth.

Qué nunca debe desarrollarse antes de otra cosa:

- Clientes avanzados (Android/iOS) antes de que el contrato del servicio que consumen esté congelado.
- Monitoring antes de tener servicios reales corriendo (no hay nada que medir).
- Optimización de rendimiento antes de que la funcionalidad esté correcta y probada.

Decisiones difíciles de revertir una vez implementadas:

- El esquema de particionamiento/replicación de Storage.
- El protocolo de comunicación de cada servicio (cambiar de SOAP a gRPC después de que 3 servicios ya lo consumen es un rediseño completo).
- La política de resolución de conflictos de Sync, si ya hay datos de usuarios reales sincronizados con la política antigua.

Roadmap elaborado para guiar la implementación del proyecto UPB-CIENTÍFICA como plataforma distribuida SOA, priorizando la reducción de retrabajo y la validación progresiva del sistema.